CryptoLocker

Internal Analysis

CSE6990 Special Topics - Software Reverse Engineering

Trent Workman

tlw342

Daniel Downs

djd133

Abstract

This paper intends to reveal the internal structure of CryptoLocker. It focuses on the internal structure, how the software executes as a whole, and detailed analysis of specific parts. Other aspects, such as the anti-debugging aspects of the code and possible bugs, are discussed as well.

Introduction

CryptoLocker is a powerful piece of ransomware that appeared in September 2013. Its goal is to make its way into an unsuspecting computer where it will attempt to contact a hidden Command and Control server over the Internet. After establishing a connection, it uses RSA and EAS encryption to encrypt the user's files [1]. There is no way for the user to decrypt the files on their own (unless they are working on a virtual machine, have a system restore set up, or have the files backed up). The malware then demands the user pay a sum of money. After the money has been sent, the malware then decrypts the files so the user can work with them.

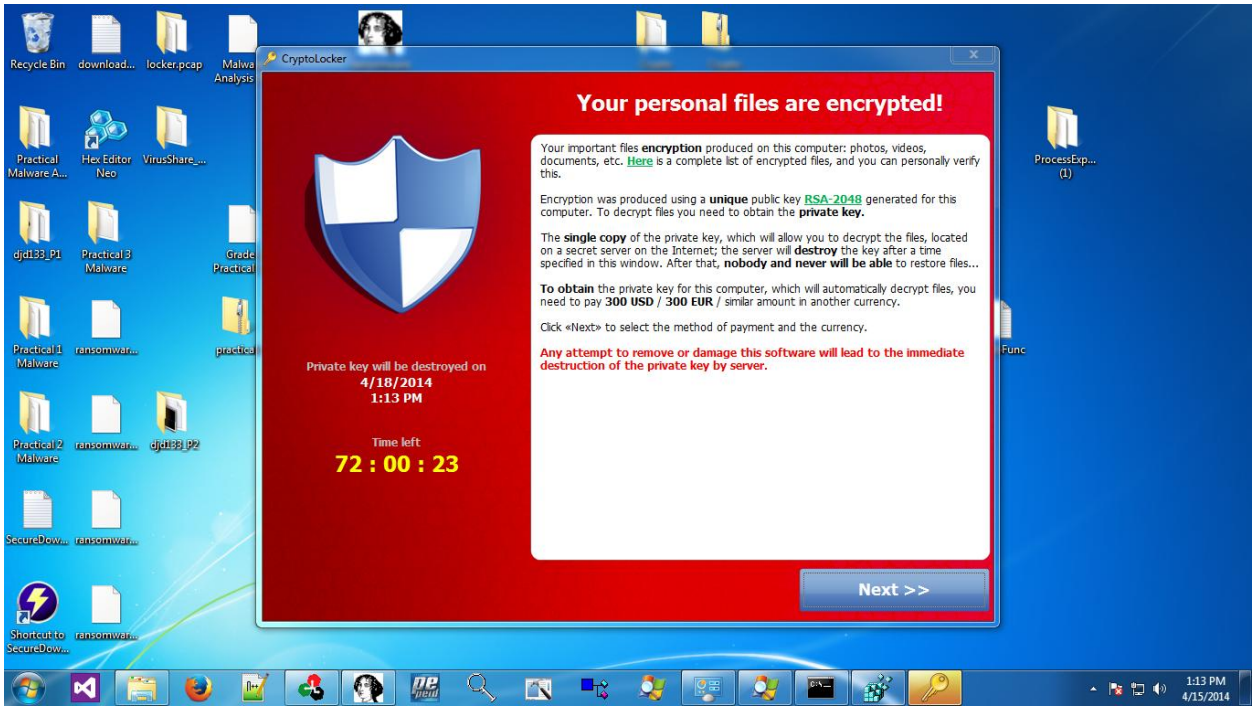

Figure 1 - GUI Displayed after a Successful CryptoLocker Infection

There has been extensive analysis done by other researchers on CryptoLocker's infection vectors, registry infection location, recovery information, encryption system and other blackbox style information. However, there has been a lack of public analysis of the internal mechanisms of CryptoLocker. The information discussed herein should allow a malware analyst to quickly learn about how CryptoLocker works from a programming perspective.

File Information

Information about the sample of CryptoLocker analyzed:

SHA256: d765e722e295969c0a5c2d90f549db8b89ab617900bf4698db41c7cdad993bb9

SHA1: 65559245709fe98052eb284577f1fd61c01ad20d

MD5: 04fb36199787f2e3e2135611a38321eb

Code Layout

The code for CryptoLocker can be divided into two large sections, here termed "Startup" and "Run" for reference. The code discussed in this section of the whitepaper is not necessarily mentioned in order of execution nor is the purpose of the code discussed (that information is addressed in the section "Run Paths"). The overarching names of the code portions as well as the subsection names have been created arbitrarily to allow for easier exposition.

"Startup" is the code that is first run when the software is launched.

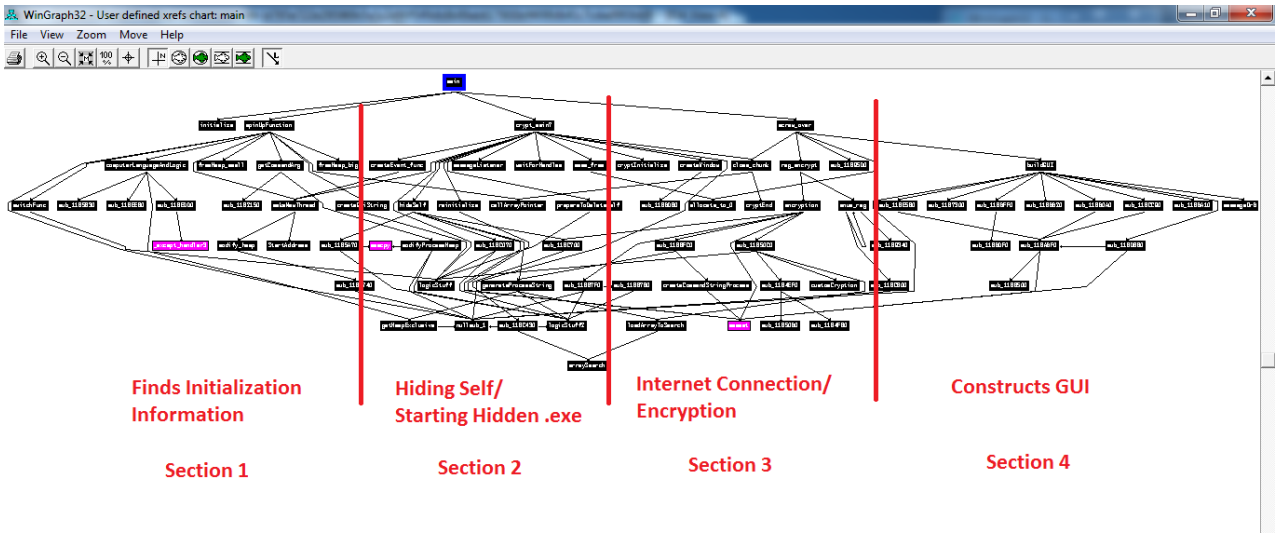

Figure 2 - "Startup" Code Section Graph (without Library Functions)

Section 1 identifies information about the infected computer. This information includes the language being used by the operating system and the Manufacturer ID (both of which are viewable via theR Window's program dxdiag). This section also parses the command arguments given via the command line when the executable was run.

Section 2 has a variety of responsibilities that it undertakes for different instances of the software: it deletes the initial executable, it creates a new copy of the executable in a different folder, it launches an alternate version of the code, and it spawns new threads for the current process.

Section 3's primary responsibility is to call the functions that display a GUI to the afflicted user, connect to the Internet, and encrypt and decrypt the files. The latter two of those responsibilities fall under the overarching "Run" section that will be discussed further in this whitepaper section. Section 3 also functions as a pseudo execution manager by keeping versions of the running executable from executing undesirable code paths.

The final part of "Startup", Section 4, is primarily focused on building and displaying the GUI to the user. However, it does involve thread messaging as well. It is worth noting that this same code can be called later in the "Run" section.

The second overarching section, “Run” is the backbone of CryptoLocker and is responsible for the serious damage this malware is known for.

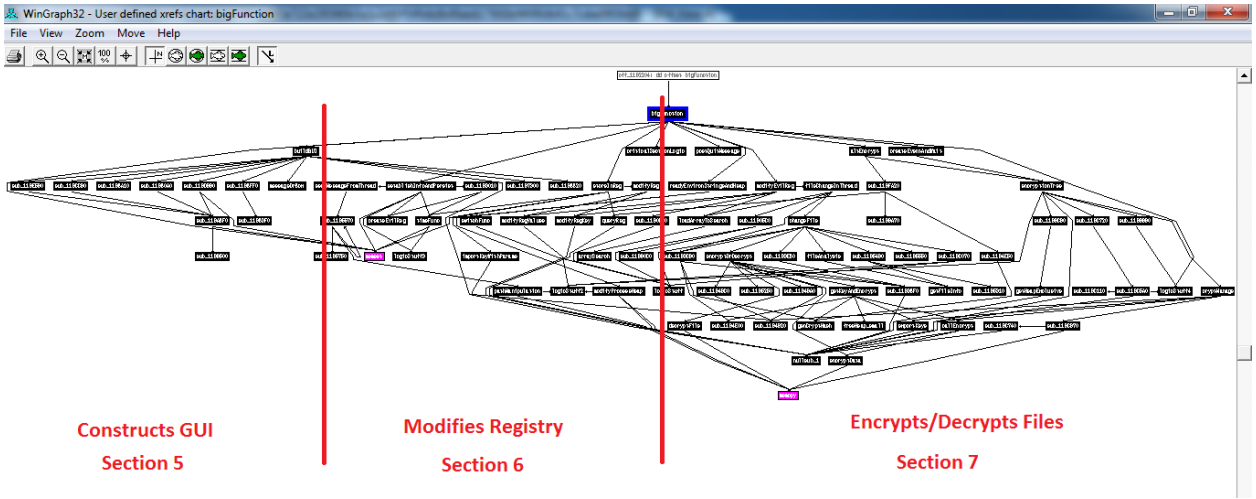

Figure 3 - "Run" Code Section Graph (without Library Functions)

Section 5 is similar to Section 4 in that it constructs the GUI presented to the user.

Section 6 is where CryptoLocker adds and modifies keys of the registry. The registry is modified to two ways. First, CryptoLocker adds a key under “HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run”. Second, it creates a new key: “HKEY_CURRENT_USER\Software\CryptoLocker\.”

Section 7 is the largest part of the code and handles both the encryption and decryption of files.

While not part of the two main sections, a secondary section, here termed “HTTPconnection” is also important.

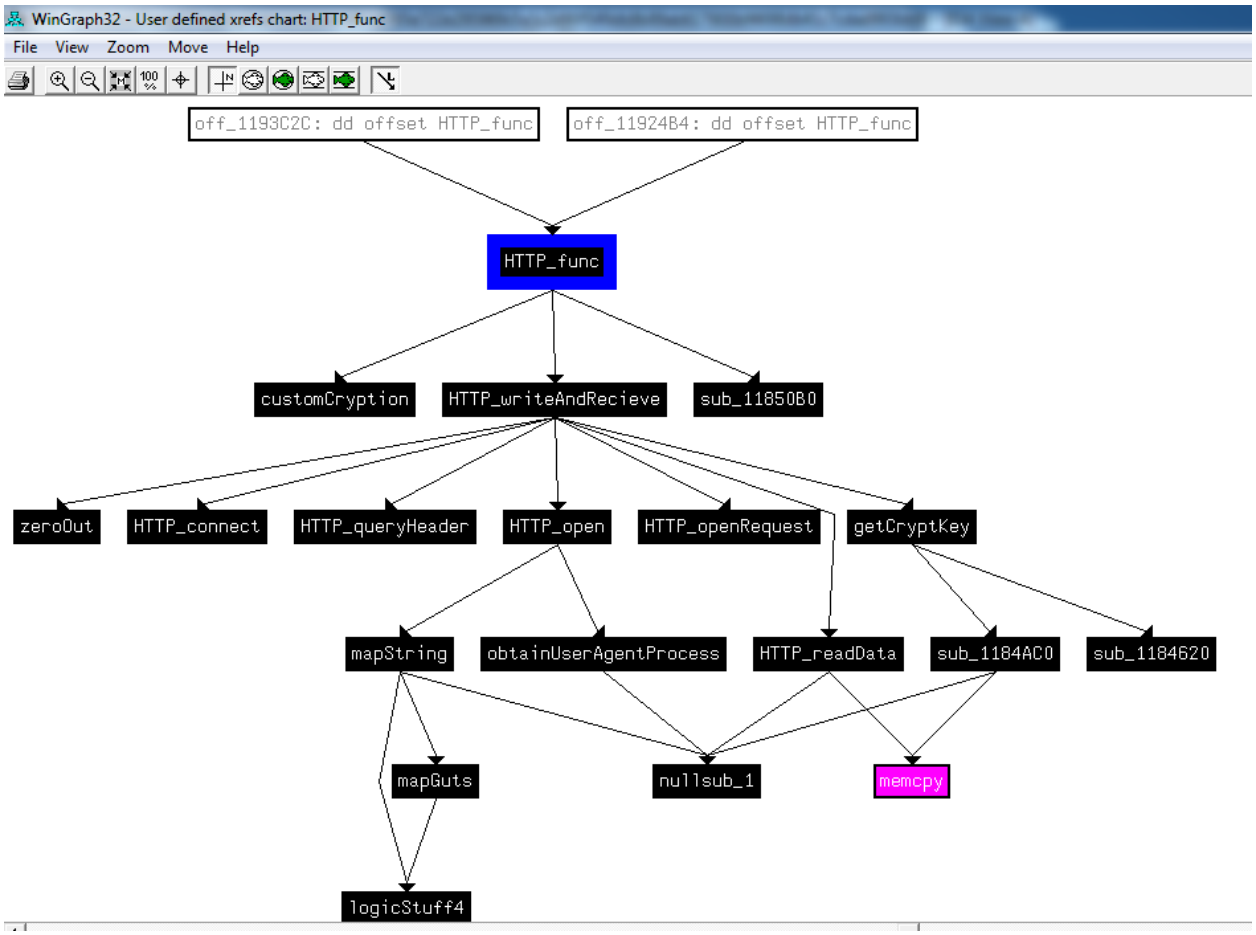

Figure 4 - "HTTPconnection" Code Section Graph (without Library Functions)

"HTTPconnection" is responsible for connecting to a valid server, receiving encryption and decryption keys, and sending payment information.

Run Paths

This section explains the primary execution path of CryptoLocker as well explanations of why it does certain actions.

After CryptoLocker is first run, it checks to see if a version of itself is already running and if so, does not execute. Assuming no other installations exist, it starts by retrieving information about the host computer and parses any command line arguments that may have been used. The

arguments can indicate if Cryptolocker should run as a watchdog to another instance instead of trying to encrypt the files. The arguments take the form of a string matching the form "/w000000XX", where X is any hexadecimal digit. Assuming the process is first instance of CryptoLocker to run, it then moves a copy of itself to the user's "AppData/Roaming/" folder and renames the copy. The program then modifies the registry to allow the newly copied executable to start up when the computer boots by adding it to "HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run". Finally, the running process starts the copied executable and exits.

The newly started code creates a watchdog instance of CryptoLocker using the aforementioned command line arguments. Should either of these two processes ever be terminated, a new copy will be started. At this point, the malware attempts to connect to a valid Command and Control server using a Doman Generation Algorithm [2]. This can be a lengthy process with the malware trying thousands upon thousands of different server names before it chances upon a valid one. After successfully connecting to the server, it receives a public encryption key. CryptoLocker uses the "GetLogicalDrives" library function to identify any connected drive, including networked drives. It then encrypts certain file types in those drives.

*.odp	*.odm	*.odc	*.odb	*.doc	*.docx	*.docm	*.wps
*.xls	*.xlsx	*.xlsm	*.xlsb	*.xlk	*.ppt	*.pptx	*.pptm
*.mdb	*.accdb	*.pst	*.dwg	*.dxf	*.dxg	*.wpd	*.rtf
*.wb2	*.mdf	*.dbf	*.psd	*.pdd	*.eps	*.ai	*.indd
*.cdr	*.dng	*.3fr	*.arw	*.srf	*.sr2	*.bay	*.crw
*.cr2	*.dcr	*.kdc	*.erf	*.mef	*.mrw	*.nef	*.nrw
*.orf	*.raf	*.raw	*.rwl	*.rw2	*.r3d	*.ptx	*.pef
*.srw	*.x3f	*.der	*.cer	*.crt	*.pem	*.pfx	*.p12
*.p7b	*.p7c	????????.jpg		????????.jpe		img_*.jpg	

Figure 5 - Files Types Encrypted by CryptoLocker [3]

As it encrypts files, it adds them to a new registry created "HKEY_CURRENT_USER\Software\CryptoLocker\." After encrypting all valid files, it creates a GUI for the user, announcing its presence and the files encrypted. The user can then pay the ransom demanded through a variety of currencies. The malware then waits for the malware author to verify the payment and to receive the order to decrypt. After receiving the command, it uses the server's private key to decrypt all encrypted files.

The GUI displayed threatens that attempting to modify the CryptoLocker installation will result in deletion of the stored private key on the server. However, though this capability was not tested, this seems unlikely. Deleting the malware will do nothing to decrypt the files already encrypted; it will only remove the option of paying the ransom and it seems unlikely that the malware authors would be willing to remove the possibility of getting paid.

CryptoLocker Internal Behavior

CryptoLocker uses a main class to hold the program's info during runtime. During analysis, parts of the class were identified and stored as a struct in IDAPro. This struct is shown in Figure 6. The "null_xxx" are values whose use has not been identified, but were initialized to a zero value. Values with the name "uninitialized_xxx" have not been used in analyzed code, and as such, the structure is unknown. Therefore, these are placed as byte arrays. The "FFFFFFFF_20" and "_77700000_254" values are named according to their value and their structure offset. These values were initialized but not used in analyzed code.

```
00000000 mainClass       struc ; (sizeof=0x4B0)
00000000 coUnitialize    dd ?
00000004 initialized     dd ?
00000008 GDI_token       dd ?
0000000C GdiStartReturn  dd ?
00000010 null_10         dd ?
00000014 null_14         dd ?
00000018 null_18         dd ?
0000001C CriticalSection dd ?
00000020 FFFFFFFF_20     dd ?
00000024 null_24         dd ?
00000028 null_28         dd ?
0000002C null_2C         dd ?
00000030 null_30         dd ?
00000034 moduleHandle    dd ?
00000038 mutexHandle     dd ?
0000003C null_3C         dd ?
00000040 null_40         dd ?
00000044 moduleFilename  db 104 dup(?)           ; char
000000AC uninitialized_AC db 416 dup(?)
0000024C mutex_GUID1     dd ?
00000250 folderNamePtr   dd ?
00000254 _77700000_254   dd ?
00000258 eventHandle     dd ?
0000025C folderName      db 104 dup(?)           ; char
000002C4 uninitialized_2C4 db 416 dup(?)
00000464 computerName    dd 10 dup(?)            ; char
0000048C null_48C        dd ?
00000490 null_490        dd ?
00000494 null_494        dd ?
00000498 currentThreadID dd ?
0000049C null_49C        dd ?
000004A0 null_4A0        dd ?
000004A4 null_4A4        dd ?
000004A8 null_4A8        dd ?
000004AC null_4AC        dd ?
000004B0 mainClass       ends
000004B0
```

Figure 6 - Main Class Structure

After loading, CryptoLocker executes its “start” function and then immediately calls the “main” function after setting up the stack. CryptoLocker allocates space for the “mainClass” and then does system setup. “SetErrorMode” controls the errors handled by the process. The 0x8003 argument to this function tells the system to not display a message box on failure to find a file, to not display Windows Error Reporting Dialog, and to not display the critical-error-handler message box. These errors are returned to the calling process and not the system, thus hiding any errors from the user. “InitializeCriticalSection” is called using a hard-coded offset. This ensures mutual exclusion from other threads and processes. CryptoLocker then gets the module handle of itself by calling “GetModuleHandle” with a NULL argument. This handle is pushed as an argument to “initialize” and “mainClass” is loaded into ECX. “initialize” is shown in Figure 7.

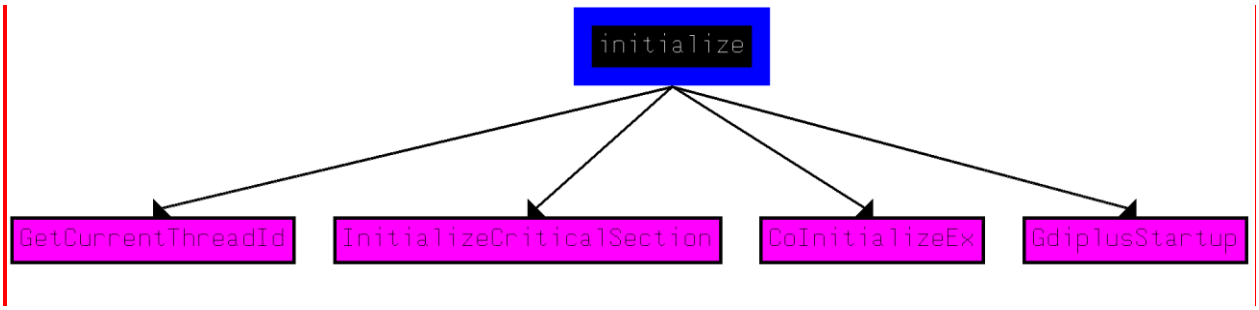

Figure 7 – "initialize" Function Graph

The "initialize" function is called, which initializes many of the values in the "mainClass." An offset to "CoUninitialize" is moved to "mainClass.coUninitialize" and the result of "CoInitializeEx" is put into "mainClass.initialized." "GdiplusStartup" is called which initialized Windows GDI+ and then sets a GDI token into "mainClass.GDI_token." If successful, a "1" is moved to "mainClass.GdiStartReturn." "InitializeCriticalSection" starts a critical section object in "mainClass.CriticalSection." Null values 10, 14, 18, 48C, 49C, 4A0, 4A4, 4A8, 3C, and 40 are initialized to zero along with "mainClass.mutexHandle." "mainClass.folderNamePtr" is zero-extended, AND-d with 0xFE, and OR-d with "2." The lower 16 bits of this result is moved back into "mainClass.folderNamePtr." "CurrentThreadId" is called and the result is moved to "mainClass.currentThreadID." "initialize" then returns back to "main."

"GetCommandLine" is called which returns the command line arguments that CryptoLocker was called with. This is pushed as an argument to "stupidInstruction" and "mainClass" is passed through ECX.

"stupidInstruction," shown in Figure 8, is called, which then calls "getLanguageInfo." This function gets the machine's language info. It does this by getting the module handle to "kernel32.dll/GetUserPreferredUILanguages." The process address of "kernel32.dll/GetUserPreferredUILanguages" is found with "GetProcAddress," and a critical section access is opened. Multiple functions to find the language are used such as "d_finds0409" and "u_409value." A table of languages is constructed and the languages are returned. In this case, "en-US," code 0x0409, was the only language. "getLanguageInfo" returns back to "stupidInstruction."

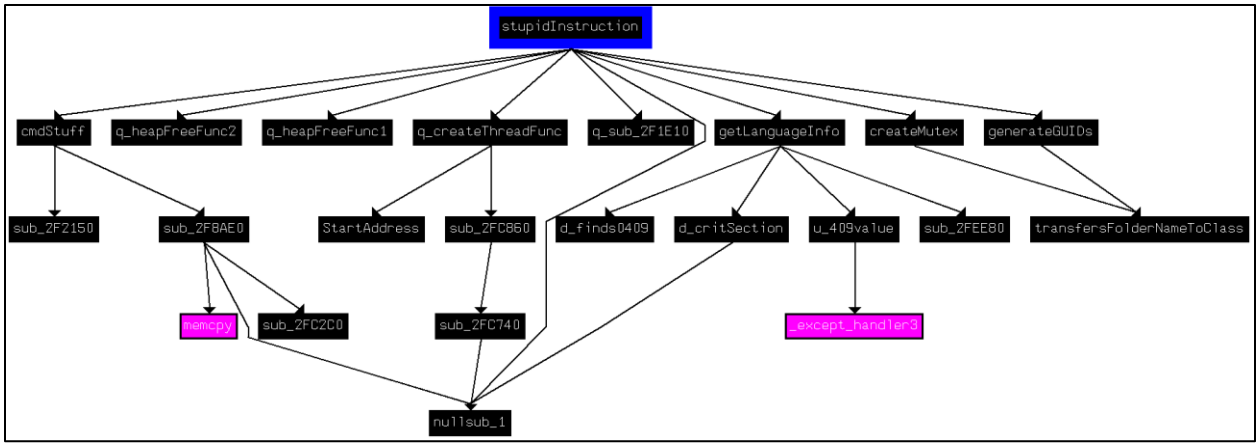

Figure 8 - "stupidInstruction" Graph (without Library Functions)

A check is made to "mainClass.initialized" and "mainClass.GdiStartReturn" to check that they are true. "cmdStuff" is called which seems to return 1 if the command line argument is currently running. "CreateEvent" is called which creates an event with a NULL name and the handle is moved to "mainClass.eventHandle." If the event is created, "GetModuleFileName" is called to retrieve the current process's filename. "generateGUIDs" is called which retrieves different system folders based on the argument passed to it. In this case, "AppData/Roaming" was returned into "mainClass.folderName." The possible folder names can be seen in Table 1. The computer name is retrieved with "GetComputerName" and put into "mainClass.computerName."

"createMutex" is called from "stupidInstruction" which will create a mutex for CryptoLocker to use located at "Global\{GUID}." This GUID comes from the folder returned from "generateGUIDs."

Folder Name	**generateGUIDs Argument**
AppData/Roaming	0
AppData/Local	1
AppData/LocalLow	2
TemporaryInternetFiles	3
Windows/Cookies	4
Users/%Username%	5

Users/%Username%/Desktop	6
Windows/Templates	7
Windows/Templates (common)	8
Program Files	9
ProgramFiles (x86)	A
ProgramFiles (x64)	B
System32	C
Windows	D

Table 1 - generateGUIDs Folders

This is the run path that has been fully analyzed by dynamic analysis. Other functions have been analyzed; however, their full functionality has not been found.

"createGUIDstring1" seems to be looking at other folders possibly returned from "generateGUIDs." It then creates a mutex for this folder and stores it in "mainClass.mutex_GUID1." If this mutex already exists, then it calls "waitForObjects." If not then the currently running process is copied into a hidden executable at "AppData/Roaming/{GUID}."

Obfuscation and Anti-Debugging Measures

Some of the following techniques could only have been implemented on purpose by the malware author with the explicit reason of hindering reverse engineering. However, others may merely be a coincidence due to the reverse engineering tools of choice and may not have been added by the malware author on purpose. Still other techniques may better serve the malware design as a whole (for example, multithreading) and only hinder research as a side effect. However, as the original author cannot be consulted on the matter, all incidences that hindered the reverse engineering of CryptoLocker are treated as deliberate anti-debugging measures and will be explained in full for the assistance of future researchers.

Redirecting a Debugger Attaching Itself to CryptoLocker

Attempting to attach a debugger (in this specific instance, Immunity Debugger) to a running process of CryptoLocker fails, as the code is immediately redirected to section of the ntdll dynamically linked library.

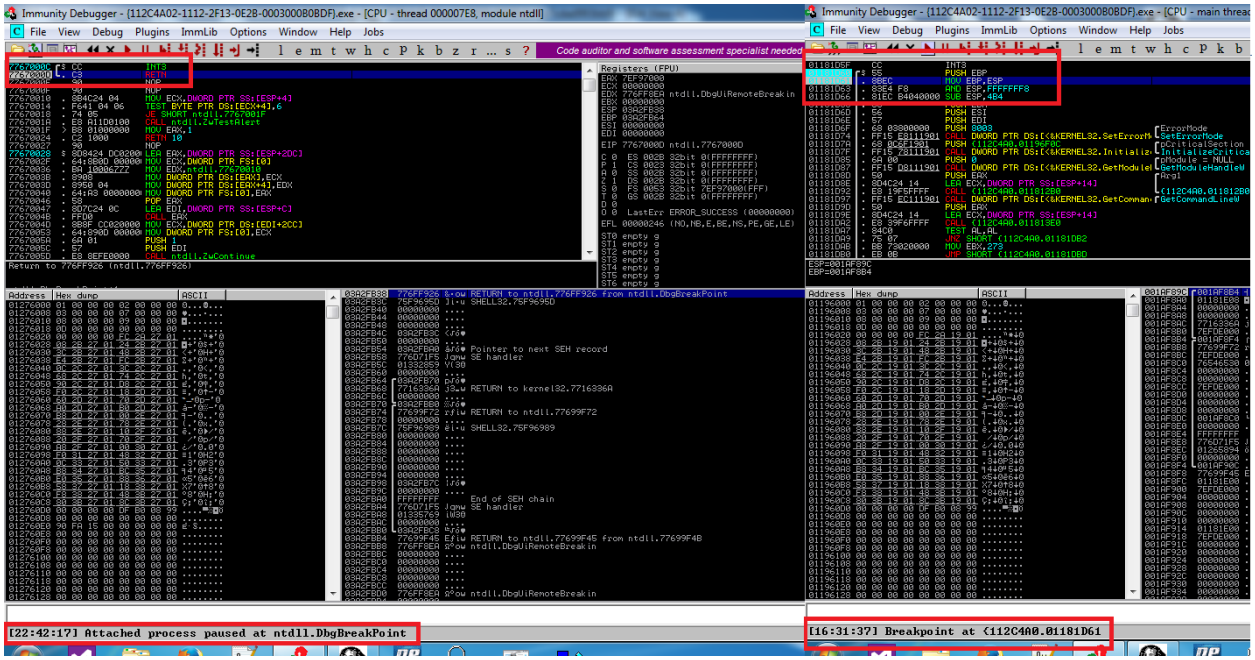

Figure 9 – a Debugger being Redirected vs. Attaching Correctly

This section immediate stops the debugging process from occurring (though the malicious processes continue). The exact method by which CryptoLocker determines that it has an attached debugger is unknown. “aadp” was a plugin that was installed for Immunity Debugger [4] in an attempt to circumvent this. When installed, it allowed for a variety of debugger detection options to be selected, after which it would attempt to keep the selected methods from successfully detecting the debugger. However, this plugin did not successfully hide the debugger from CryptoLocker. It was later found out that this plugin was developed for 32 bit operating systems which may have been the cause of the failure.

To debug the “meat” of the code, after the copied .exe has been started, attempt to attach to the new process. After it is redirected, kill the process tree. Then, use the recent files under the “File” tab to start the process in Immunity Debugger and debug successfully.

Code Run Conditions

The primary encrypting that takes place (a small amount takes place at the beginning of the code) only happens after an internet connection has successfully been established. This means that if the researcher is avoiding an internet connection (as is prudent when reverse engineering malware), the researcher will have to choose whether to see the code run only a portion of its functionality, to make CryptoLocker think it has a valid Internet connection or to build a valid Command and Control (C&C) server that can connect to CryptoLocker and then patch the executable to only access that server. As pointed out before, CryptoLocker uses a Domain Generation Algorithm to find valid C&C servers, so this functionality would also have to be removed.

Attempting to make CryptoLocker think it has a valid Internet connection is trivial (changing a register in Immunity Debugger to change a “jmp” path) but generating a valid encryption key that it should have received from a correct Internet connection is not. As a result, if the researcher does not debug with an open connection, there is no simple solution to this issue and each researcher will have to choose the best solution for their situation.

However, even if the researcher is running a virtual machine and can expose their machine without risk, they still face an even greater hurdle: file decryption only takes place after payment has been validated. Since CryptoLocker will not decrypt the files until the malware author directs it to, there is no way for a researcher to perform dynamic analysis with either paying the ransom or building their own C&C server.

Calling a Function Pointer instead of the Function Location

A common method for quickly seeing the entire layout of a program's code is to use the graph function of IDA Pro. However, if the address of a call is determined at run time (for example by loading the address of a function from an array), the graph command will not continue into that call, and portions of the complete graph will be lost.

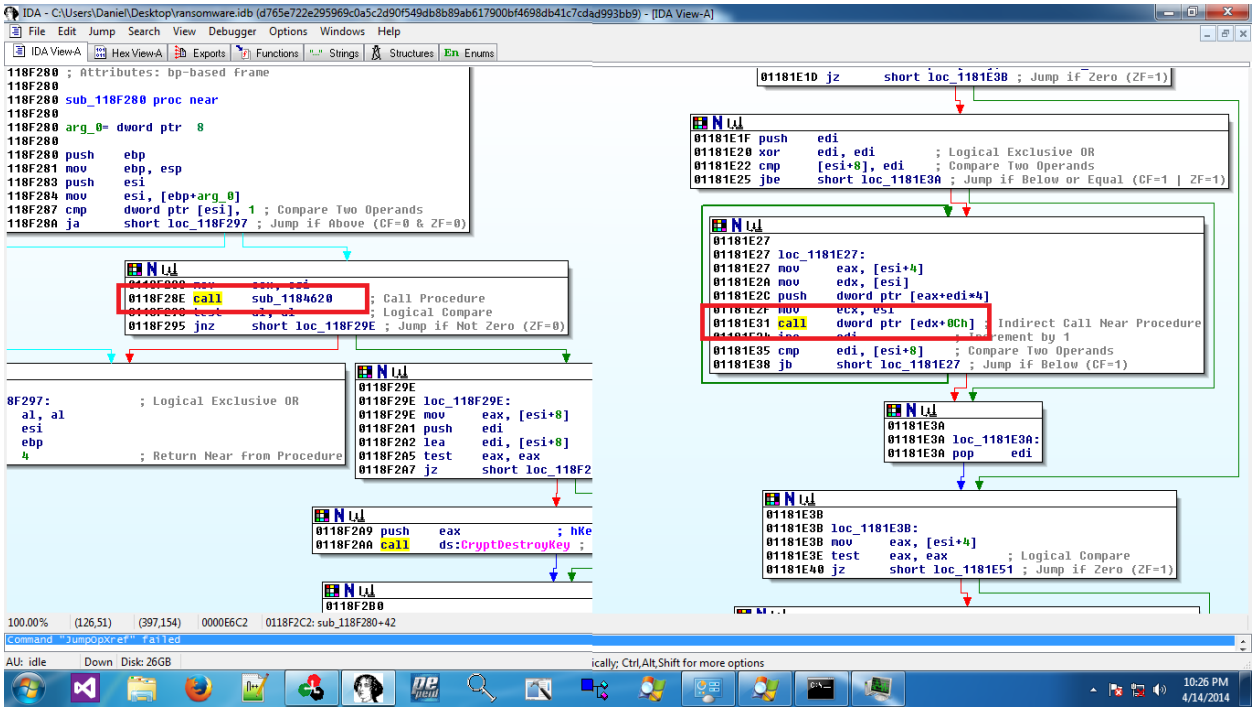

Figure 10 - A Function Call Determined at Compile Time vs. a Call Determined at Runtime

There is no perfect solution to this problem as IDA Pro cannot recursively trace up a graph, only down. A partial solution is to first identify a function not seen in the graph (all functions will be listed in the IDA Pro "Functions" tab), jump to said function, and check the xrefs to the function.

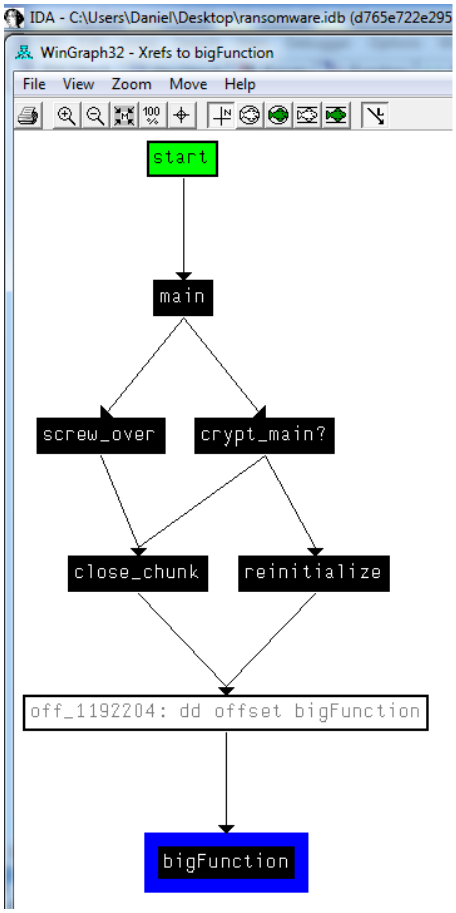

Figure 11 - An IDA Pro "xref to" Graph for a Dynamically Called Function

This will show the function tree that results in a call to the function, even if the function call is determined at runtime.

Moving and Hiding of Executable

After the executable is run, it renames itself, copies itself to a new location, and deletes the original copy. Since the original does not do much work other than the copying and launching of the copy, and since CryptoLocker actively checks for a debugger attaching to a running process, it can be difficult to see the new executable running the important code. To counter this, after the new executable has been generated and launched, killing the running process tree (to kill the watchdog process as well) and relaunching the executable from Immunity Debugger will allow researchers to view the code in its completion.

Multithreaded/Multiprocess Code Construction

It is unlikely that the malware author used multiple threads and processes for the explicit purpose of making reverse engineering challenging as it is a difficult solution that does not reap much counter analysis benefit, especially with more easily implemented measures. However, the fact remains that when combined with previous anti-debugging techniques, especially checking for the presence of debuggers, it is an effective means of complicating reverse engineering. Dealing with the multiple processes can be mitigated via the solutions provided in the "*Checking for the Presence of Debuggers*" subsection as well as debugging the executable with the command line arguments (in the case of the watchdog). Doing so will reveal the code paths that are being executed.

Dealing with multiple threads is a different matter. While individual threads will not check for debuggers, they will quickly and quietly execute code if not closely monitored, especially after a call to "WaitForMultipleObjects". One possible methods of avoiding this is to simultaneous monitor each thread as it appears. However, it is noticed that in Immunity Debugger, threads may not appear in the "Thread" window until after a "WaitForMultipleObjects" call, at which point they will appear and quickly move through the code. A second, more recommended, choice would be to do light static analysis of the entire CryptoLocker, identifying several important functions, and then setting breakpoints on them. This way, even if another thread gets to that function unbeknownst to the researcher, execution will stop and allow the researcher to examine the situation.

Unknown OpCodes in Immunity Debugger

When CryptoLocker is being run dynamically in Immunity Debugger, some x86 assembly OpCodes are spread across multiple lines instead of being grouped together. It should be noted that even when spread out like this, the code is still valid and will execute correctly as though it was a single OpCode.

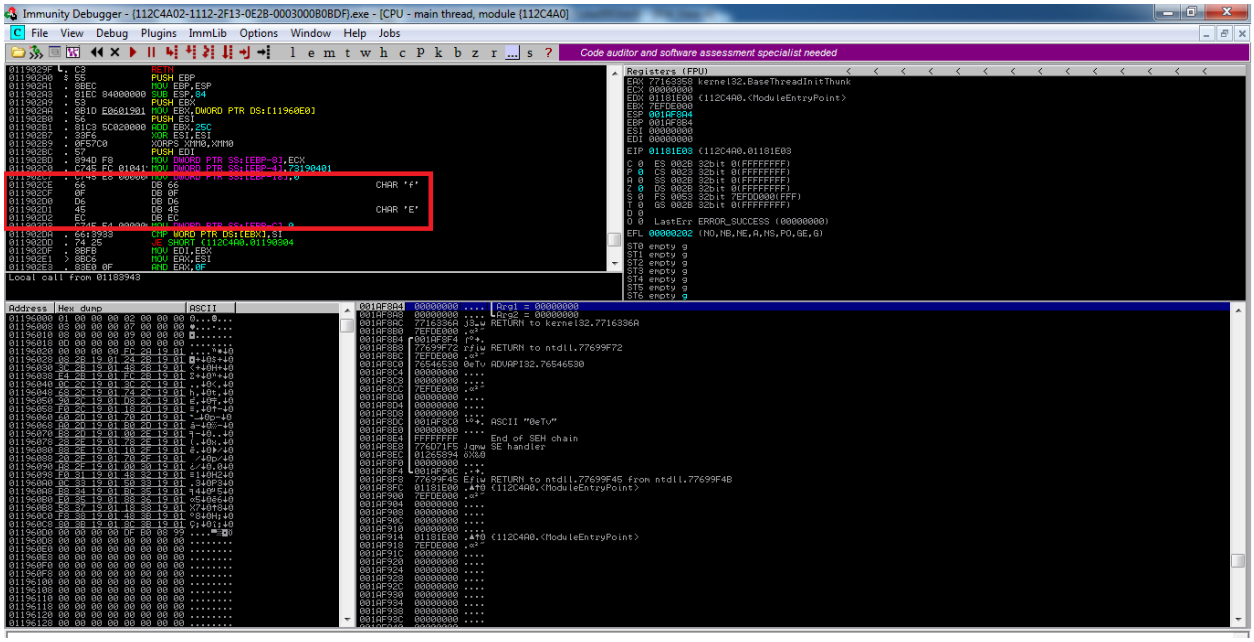

Figure 12 - A Single OpCode Spread across Multiple Lines

Immunity Debugger will not step over this code, as it is unrecognized and they will not step over unrecognized code by default. Luckily, this problem is easily remedied. Immunity Debugger has an option to allow stepping into unrecognized code (Options -> Security) which then allows the researcher to continue work.

Address Space Layout Randomization

CryptoLocker uses Address Space Layout Randomization (ASLR) to shift address bases on every launch. This makes developing exploits for Crytolocker more difficult. While this is the least obtrusive anti-debugging technique, it can still be annoying, especially if dynamic analysis (for example with Immunity Debugger) is used in conjunction with static analysis (for example with IDA Pro). Fortunately, this is fairly simple to stop. Patching the executable to zero out the ASLR bit will keep the executable from rebasing itself. Finding the offset to this bit can be done with a tool such as PEView [5] by looking at the IMAGE_OPTIONAL_HEADER under IMAGE_NT_HEADER.

Possible Bugs and Weaknesses in CryptoLocker

Use of “GetModuleHandle” instead of “GetModuleHandleEx” library function may cause a race condition.

Conclusion

The power of CryptoLocker is impressive. Persistent and non-reversible, it has infected users across the globe and extorted millions of dollars.

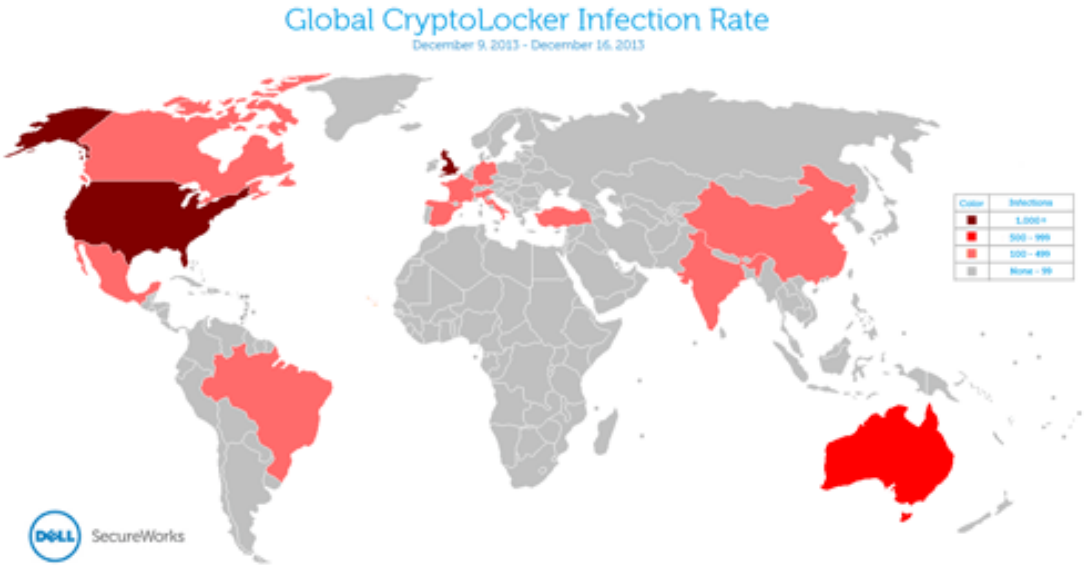

Figure 13 - CryptoLocker Global Infection Rates [6]

Recently, other ransomware has appeared that has been inspired by CryptoLocker. For example CryptoLocker 2.0, a .NET version with expanded capabilities has been created by copycats [7]. These kinds of “social engineering” based malware are likely to continue being deployed for the foreseeable future and continued in-depth analysis will be the key to defending against and stopping them.

References

[1] L. Abrams, CryptoLocker Ransomware Information Guide and FAQ, Available: http://www.bleepingcomputer.com/virus-removal/CryptoLocker-ransomware-information#CryptoLocker

[2] S. Yazdi, M. Joven, and L. Huang, A Closer Look at Cryptolocker's DGA, Available: http://blog.fortinet.com/A-Closer-Look-at-Cryptolocker-s-DGA/

[3] P. Ducklin, Destructive malware "CryptoLocker" on the loose – here's what to do, Avaliable: http://nakedsecurity.sophos.com/2013/10/12/destructive-malware-cryptolocker-on-the-loose/

[4] nahuelr, Anti-Anti-Debugger Plugins, Available: https://code.google.com/p/aadp/

[5] Aldeid, PEView, Available: http://www.aldeid.com/wiki/PEView

[6] K. Jarvis, CryptoLocker Ransomware, Available: http://www.secureworks.com/cyber-threat-intelligence/threats/cryptolocker-ransomware/

[7] J. Dunn, CryptoLocker 2.0 turns into worm that spreads via USB drives, Available: http://news.techworld.com/security/3495444/cryptolocker-20-turns-into-worm-that-spreads-via-usb-drives/

Microsoft MSDN, Available: http://msdn.microsoft.com/en-us/library/windows/desktop/bg125389.aspx

M. Sikorski and A. Honig, "Anti-Debugging," in *Practical Malware Analysis*, San Francisco:No Starch Press, 2012, ch. 16, pp. 351 – 368